

Control Variates for a GARCH Model

Monte Carlo Project – Oral Support

Adam BERNABEU, Tom BOURDAREAU, Nicolas GRIMALDI

May 4, 2026

Reference paper:

- MCMC for GARCH posterior inference with variance reduction ideas.
- Main tool: **control variates** to reduce Monte Carlo error.
- Our implementation follows the experiment protocol:
 - ▶ Random Walk Metropolis for a GARCH(1,1) posterior.
 - ▶ Comparison of estimators (plain MCMC vs corrected estimators).
- Second paper extends to high-dimensional control variates via Lasso.

Implement a Random Walk Metropolis sampler
for a GARCH(1,1) posterior, first on simulated data,
then on real exchange-rate log-returns.

Q1 – Mathematical setup

$$r_t = \sqrt{h_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 1),$$

$$h_t = \omega_1 + \omega_2 r_{t-1}^2 + \omega_3 h_{t-1}, \quad \omega_1 > 0, \omega_2, \omega_3 \geq 0, \omega_2 + \omega_3 < 1.$$

$$\pi(\omega \mid r_{1:T}) \propto L(r_{1:T} \mid \omega) p(\omega).$$

$$\alpha(\omega, \omega') = \min\left(1, \frac{\pi(\omega' \mid r)}{\pi(\omega \mid r)}\right).$$

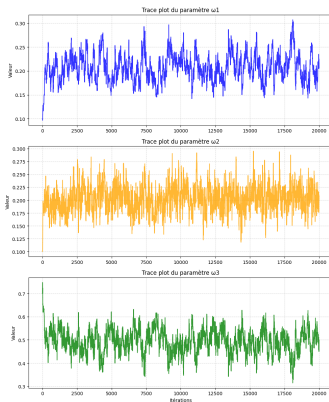
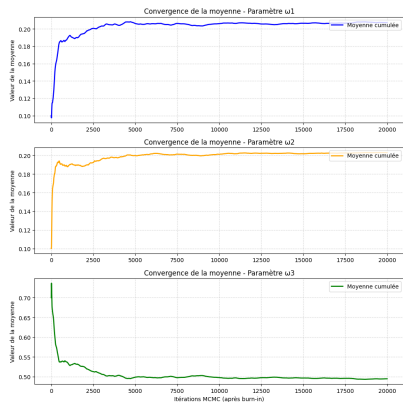
- Random-walk proposal: $\omega' = \omega + \eta$, $\eta \sim \mathcal{N}(0, \Sigma)$.

Q1 – Core sampler (code extract)

```
def metropolis_hastings(r, iters=10000, scale=1.0):
    chain = np.zeros((iters, 3))
    current_w = np.array([0.1, 0.1, 0.7])
    current_ll = log_likelihood(current_w, r)
    current_lp = log_prior(current_w)
    for i in range(iters):
        proposal_w = current_w + rng.normal(0, step_sizes, 3)
        proposal_lp = log_prior(proposal_w)
        if proposal_lp > -np.inf:
            proposal_ll = log_likelihood(proposal_w, r)
            log_ratio = (proposal_ll + proposal_lp) - (current_ll + current_lp)
            if np.log(rng.random()) < log_ratio:
                current_w = proposal_w
                current_ll = proposal_ll
                current_lp = proposal_lp
    chain[i] = current_w
```

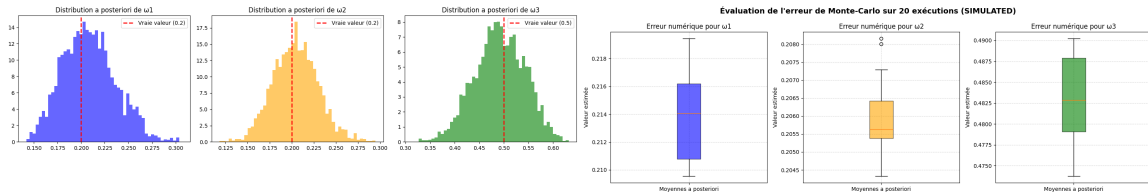
- Prior constraints enforce positivity and stationarity ($\omega_2 + \omega_3 < 1$).

Q1 – Simulated-data diagnostics



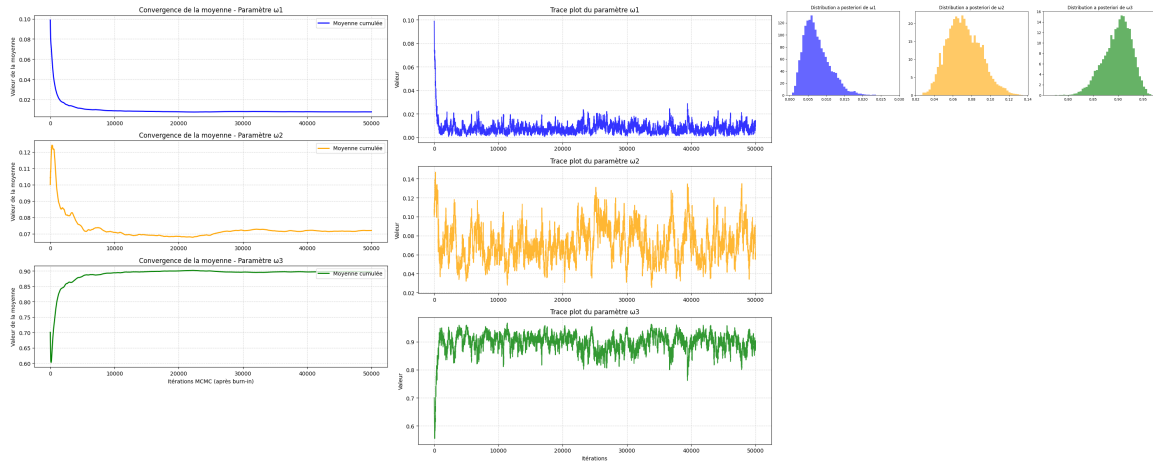
- Trace plots stabilize after burn-in with the tuned proposal scale.

Q1 – Posterior distributions and repeatability



- Repeated runs show visible Monte Carlo variability.
- This motivates control variates in Q2–Q3.

Q1 – Real-data run (EUR/USD log-returns)



- Same pipeline transfers to market data.
- Real data confirms the need for variance reduction.

Use first-order control variates

and estimate coefficients by linear regression
to reduce estimator variance.

Q2 – Math for degree-1 control variates

Target quantity:

$$\mu_f = \mathbb{E}_\pi[f(\omega)].$$

First-order corrected estimator:

$$\tilde{f}(\omega) = f(\omega) + a^\top z(\omega), \quad z(\omega) \approx -\frac{1}{2} \nabla \log \pi(\omega \mid r).$$

Choose a by regression over MCMC samples:

$$f(\omega^{(i)}) = \beta_0 + b^\top z(\omega^{(i)}) + \varepsilon_i, \quad a = -b.$$

$$\hat{\mu}_f^{cv} = \frac{1}{N} \sum_{i=1}^N \tilde{f}(\omega^{(i)}).$$

Q2 – First-order ZV control variates

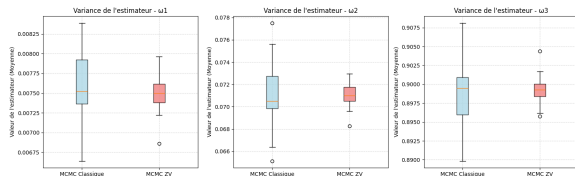
```
@njit
def compute_garch_gradient_fast(w, r):
    # returns  $Z(w) = -0.5 * \text{grad} \log \pi(w)$ 
    ...

# Regression correction for each target function f
X = Z_run
Y = f_samples
reg = LinearRegression().fit(X, Y)
f_corrected = Y - X @ reg.coef_
```

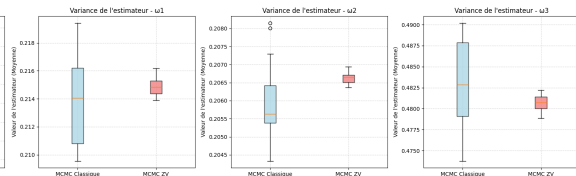
- Linear model gives the best coefficients in least-squares sense.
- Cost is moderate for degree-1 features.

Q2 – Variance reduction results

Comparaison de la dispersion des estimateurs (MCMC vs ZV) sur 20 chaînes (REAL)



Comparaison de la dispersion des estimateurs (MCMC vs ZV) sur 20 chaînes (SIMULATED)



- Boxplots show narrower dispersion than naive MCMC means.
- Numerical-error assessment is done over repeated runs.

Use a larger set of control variates (degree 2),
then adapt the Lasso-based approach
to handle feature explosion.

Q3 – Degree-2 controls and Lasso

Degree-2 basis (schematically):

$$\phi(\omega) = [z_1, z_2, z_3, \omega_j z_i - \delta_{ij}/2]_{i,j=1}^3 \in \mathbb{R}^{12}.$$

High-dimensional issue (for larger models):

$$p \uparrow \Rightarrow X^\top X \text{ inversion unstable/costly.}$$

Lasso correction:

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2N} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1,$$

$$\hat{\mu}_f^{lasso} = \frac{1}{N} \sum_{i=1}^N \left(f_i - (X\hat{\beta})_i \right).$$

Q3 – Why Lasso? (code extract)

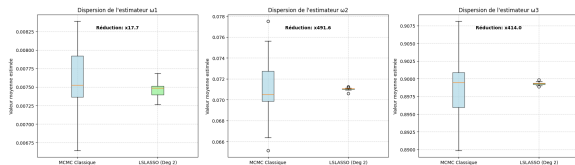
```
# Degree-2 design: 3 first-order + 9 second-order terms
features = []
for i in range(3):
    features.append(Z_run[:, i])
for i in range(3):
    for j in range(3):
        delta = 0.5 if i == j else 0.0
        features.append(W_run[:, j] * Z_run[:, i] - delta)
X_design = np.column_stack(features)

lasso = LassoCV(cv=5).fit(X_design, Y)
Y_corr = Y - X_design @ lasso.coef_
```

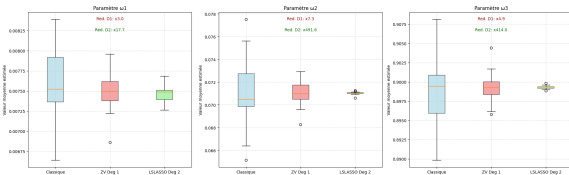
- Feature count grows quickly with polynomial order.
- Lasso keeps only useful controls and stabilizes estimation.

Q3 – LSLASSO vs naive regression

Comparaison de l'Erreur Numérique : MCMC Classique vs LSLASSO (REAL)



Comparaison de la réduction de variance (Degré 1 vs Degré 2 Lasso) (SIMULATED)



- LSLASSO remains competitive/improved when controls are many.
- Better bias-variance trade-off than one-shot OLS on large designs.

Part 4 – Bonus Question

Is standard linear regression fully valid on MCMC draws?

Not exactly: draws are dependent.

We tested practical fixes: **thinning** and **batching**.

Q4 – Dependence in MCMC (math view)

For a chain (X_t) , CLT involves integrated autocorrelation:

$$\sqrt{N}(\bar{f}_N - \mu_f) \Rightarrow \mathcal{N}(0, \sigma_f^2 \tau_{\text{int}}),$$

$$\tau_{\text{int}} = 1 + 2 \sum_{k \geq 1} \rho_k.$$

Effective sample size:

$$\text{ESS} \approx \frac{N}{\tau_{\text{int}}}.$$

Practical fixes:

$$\text{Thinning: } X'_m = X_{mk}, \quad \text{Batching: } B_m = \frac{1}{b} \sum_{t=(m-1)b+1}^{mb} X_t.$$

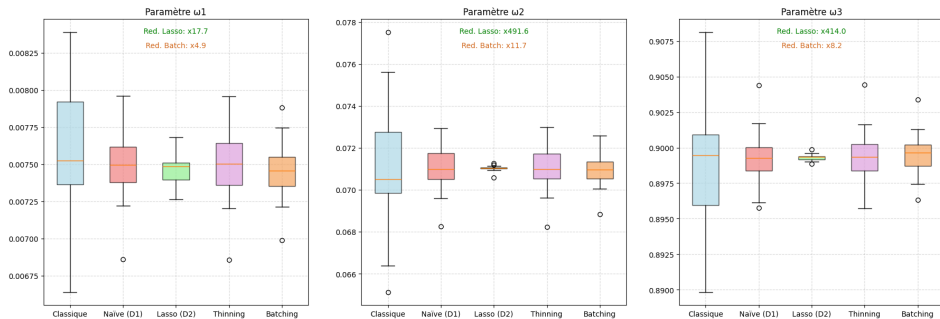
Q4 – Dependence mitigation (code extract)

```
def apply_thinning(chain, k):  
    return chain[::k]  
  
def apply_batching(chain, batch_size):  
    n_batches = len(chain) // batch_size  
    truncated = chain[:n_batches * batch_size]  
    return truncated.reshape(n_batches, batch_size, 3).mean(axis=1)
```

- Thinning lowers autocorrelation by subsampling.
- Batching averages local dependence and approximates iid blocks.

Q4 – Empirical comparison

Comparaison de la Validité de la Régression MCMC (Thinning vs Batching)



- Dependence-aware variants can improve stability of corrections.
- Practical choice depends on ESS/cost trade-off.

Takeaways

- RW-Metropolis works for simulated and real GARCH data.
- Control variates (degree 1) clearly reduce Monte Carlo error.
- Degree-2 controls become expensive; LSLASSO is a scalable fix.
- Bonus: accounting for MCMC dependence is important in practice.